WEEK 11: NP-COMPLETENESS

Advanced Algorithms

DECISION VS SEARCH

THIS COURSE SO FAR: FIND THINGS

Almost every algorithm we built this semester solves a **search problem**:

- **W3:** find a maximum independent set of a tree
- **W5–6:** find a minimum spanning tree
- **W8:** find a maximum s-t flow
- **W10:** find a maximum matching *and* a König cover certifying it

Today we study problems where we believe *no* efficient algorithm exists. For that theory, it is cleaner to strip a problem down to a yes/no question.

DECISION PROBLEMS

Definition A **decision problem** asks a yes/no question about the input. A **search problem** asks to produce an object.

- **Search:** find a maximum matching of G.
- **Decision:** given G and a number k, is there a matching of size at least k?

The decision version is **never harder**: find a maximum matching M, then answer yes if and only if $|M| \geq k$.

DECISION PROBLEMS

SEARCH	DECISION
find a maximum matching of G	does G have a matching of size $\geq k$?
find a minimum vertex cover of G	does G have a vertex cover of size $\leq k$?
find a satisfying assignment of φ	is φ satisfiable?

Same question in two costumes — the decision form keeps the essential difficulty.

WHY COMPLEXITY THEORY USES DECISION PROBLEMS

- Uniform output: every problem returns one bit. Problems become *sets of yes-instances*, easy to compare and to chain reductions through.
- Hardness transfers upward: if the decision version is hard, the search version is at least as hard.
- And usually nothing is lost in the other direction…

DECISION $\Rightarrow$ SEARCH (USUALLY)

Suppose a magic box answers "does G have a matching of size $\geq k$?" How do we *find* a maximum matching?

- Binary search on k to learn the optimum value k^*.
- Pick any edge e. Ask: does $G - \{e\text{'s endpoints}\}$ have a matching of size $\geq k^* - 1$?
- If yes: commit to e and recurse. If no: delete e and try another edge.

DECISION $\Rightarrow$ SEARCH (USUALLY)

Polynomially many questions in total — this pattern (*self-reducibility*) works for SAT, independent set, …

> **TAKEAWAY** For the problems in this lecture, decision and search stand or fall together. We study the decision versions.

THE CLASS NP

REMEMBER THIS FROM WEEK 2?

The SAT problem: the input is an AND of ORs of variables and their negations.

$$(x_3 \vee x_2 \vee x_4) \wedge (\neg x_4 \vee x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_4 \vee \neg x_1)$$

Is there a Boolean assignment to the variables making the formula true?

Here, $x_1 = 1,\ x_2 = 0,\ x_3 = 1,\ x_4 = 0$ works — check each clause!

In Week 2 we used SAT as the anchor for fine-grained lower bounds (ETH, SETH). Today it anchors a coarser, older, and even more influential theory.

HARD TO FIND, EASY TO CHECK

Notice the asymmetry:

- **Finding** a satisfying assignment: best known algorithms take exponential time ($O(2^{0.39n})$ for 3SAT [HKZZ19]).
- **Checking** a proposed assignment: scan the clauses — linear time.

[HKZZ19] Hansen, Kaplan, Zamir and Zwick, "Faster k-SAT Algorithms Using Biased-PPSZ", STOC 2019.

HARD TO FIND, EASY TO CHECK

A satisfying assignment is a certificate: a short proof that the answer is yes, verifiable quickly.

NP is exactly the class of problems with this “easy to check” property.

NP VIA VERIFIERS

Definition A decision problem L is in **NP** if there is a polynomial-time **verifier** V and a polynomial p such that for every input x:

- x is a yes-instance $\iff$ there exists a **certificate** c with $|c| \leq p(|x|)$ such that $V(x, c)$ accepts.

Two-sided requirement: yes-instances have *some* convincing certificate; for no-instances, *no* certificate fools V.

V never has to find c — it only judges a c handed to it. Think of c as a completed puzzle and V as the person checking the rules.

CERTIFICATES FOR OUR PROBLEMS

Problem	Certificate	Verifier checks
SAT	an assignment	every clause has a true literal
Independent set $\geq k$	the set S	$\lvert S\rvert \geq k$ and no edge inside S
Vertex cover $\leq k$	the set C	$\lvert C\rvert \leq k$ and every edge has an endpoint in C

Each check is a simple loop over the input — comfortably polynomial time.

$\mathbf{P} \subseteq \mathbf{NP}$

Fact $\mathrm{P} \subseteq \mathrm{NP}$: if L is decidable in polynomial time, take the empty certificate and let $V(x, c)$ just solve x itself.

The converse is the most famous open problem in computer science:

$$\mathrm{P} \overset{?}{=} \mathrm{NP}$$

THE BIG QUESTION

Is checking a proof really no easier than finding one? A million-dollar Clay Millennium problem; the wide belief is $\mathrm{P} \neq \mathrm{NP}$.

Week 2's ETH is a stronger, quantitative version of $\mathrm{P} \neq \mathrm{NP}$: 3SAT needs time 2^{cn} for some constant $c > 0$, not just super-polynomial time.

PROVING A NEGATIVE

Consider the complement question:

PUZZLE "G has **NO** independent set of size $\geq k$." What short certificate would convince you of that?

- Listing all $\binom{n}{k}$ candidate sets and their flaws — exponentially long.

DEFINITION A decision problem is in **co-NP** if its complement is in NP: every *no*-instance has a short certificate that a polynomial-time verifier can check.

PROVING A NEGATIVE

- "No independent set of size $\geq k$" is in co-NP. Is it in NP? No short proof of absence is known; NP $\overset{?}{=}$ co-NP is open too.
- Contrast Week 10: for *bipartite* graphs, König gave us short certificates in both directions (a matching vs a vertex cover of equal size). Duality put bipartite matching in NP $\cap$ co-NP.
- Certificates alone do not give an algorithm: we found the matching with max flow.

POLYNOMIAL-TIME REDUCTIONS

REDUCTIONS AS TRANSLATIONS

You already reduce all the time:

- **W2:** LIS as a special case of LCS (sort, then LCS).
- **W6:** minimax paths answered by the paths in an MST.
- **W8:** bipartite matching solved by translating to max flow.

A reduction is a *translation* between problem languages: rewrite a question about A as a question about B, so a B-solver answers it.

So far we used translations to **solve** problems. Today we run them in reverse — to **spread hardness**.

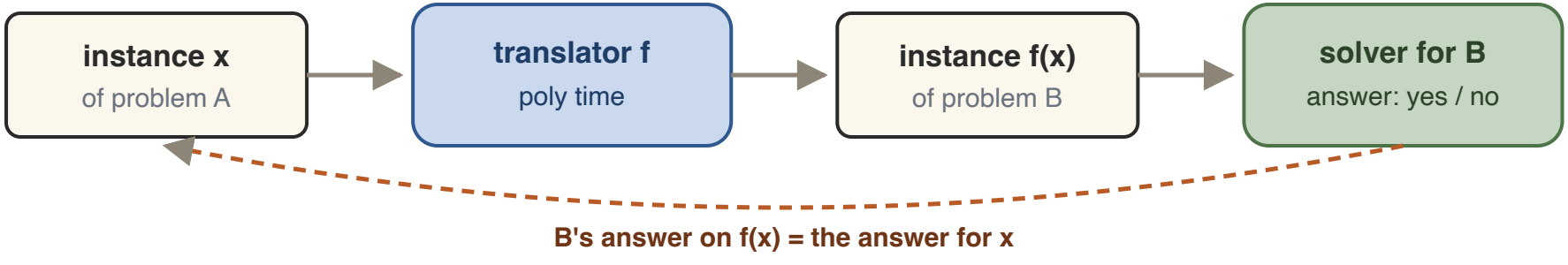
DEFINITION

Definition $A \leq_p B$ ("A reduces to B in polynomial time") if there is a polynomial-time computable function f mapping instances of A to instances of B such that for every x: x is a yes-instance of $A \iff f(x)$ is a yes-instance of B.

Read $A \leq_p B$ as: "*A is no harder than B (up to polynomial factors)*".

Note both directions of the $\iff$ matter: the translation must preserve yes *and* no answers.

THE REDUCTION PIPELINE



An algorithm for B plus a translation f gives an algorithm for A.

WHAT A REDUCTION PROVES

Suppose $A \leq_p B$ — the same fact reads two ways:

EASINESS FLOWS BACKWARD If B has a polynomial-time algorithm, so does A: translate, then solve.

HARDNESS FLOWS FORWARD Contrapositive: if A has no polynomial-time algorithm, neither does B. (Today.)

And reductions compose: $A \leq_p B$ and $B \leq_p C$ give $A \leq_p C$ — polynomials of polynomials are polynomials.

THE CLASSIC PITFALL

CAREFUL To prove that B **is hard**, reduce **FROM** a known hard problem **TO** B: $\underbrace{A}_{\text{known hard}} \leq_p \underbrace{B}_{\text{your problem}}$. Reducing B to a hard problem proves nothing — every problem in NP reduces to SAT!

- $A \leq_p B$ says "B is at least as hard as A" — the harder problem sits on the *right*.

NP-COMPLETENESS

THE HARDEST PROBLEMS IN NP

Definition A problem B is **NP-hard** if $A \leq_p B$ for *every* $A \in$ NP.

Definition B is **NP-complete** if B is NP-hard *and* $B \in$ NP.

THE HARDEST PROBLEMS IN NP

NP-complete problems are the summit of NP: solve any one of them in polynomial time and *everything* in NP collapses into P — so if $\mathrm{P} \neq \mathrm{NP}$, none of them has a polynomial-time algorithm.

But wait — "every problem in NP reduces to B" is a claim about infinitely many problems. How could we ever prove such a thing?

COOK–LEVIN

Theorem (Cook 1971, Levin 1973) SAT is NP-complete.

SAT $\in$ NP: the assignment is the certificate (we checked). The remarkable part is NP-hardness: *every* problem with a polynomial-time verifier reduces to SAT.

This finally explains Week 2's remark: "solving 3SAT in polynomial time is equivalent to $\mathrm{P} = \mathrm{NP}$."

We won't prove Cook–Levin — but the idea deserves one slide.

WHY SAT? COMPUTATION IS LOCAL

Take any $A \in$ NP with verifier V. Fix an input x.

- Write the entire run of $V(x, c)$ as a *tableau*: one row per time step, recording the machine's full state. Polynomial time $\Rightarrow$ polynomially many rows and columns.
- Each cell of the tableau is determined by a few neighbouring cells in the row above — computation is **local**.

WHY SAT? COMPUTATION IS LOCAL

- Introduce a Boolean variable per cell value; each local consistency rule becomes a few clauses; add clauses saying "the final state accepts".
- The certificate bits c stay as free variables. The formula is satisfiable $\iff$ some certificate makes V accept $\iff$ x is a yes-instance.

One big (but polynomial-size) CNF formula simulates the whole computation.

SAT $\leq_p$ 3SAT

3SAT: every clause has exactly 3 literals. To reduce SAT to 3SAT, transform each clause:

- **Short clauses:** pad by repeating a literal — $(\ell_1 \vee \ell_2)$ becomes $(\ell_1 \vee \ell_2 \vee \ell_2)$.

SAT $\leq_p$ 3SAT

- **Long clauses:** split, chaining fresh variables through: $(\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4 \vee \ell_5)$ becomes, with new variables y_1, y_2:

$$(\ell_1 \vee \ell_2 \vee y_1) \wedge (\neg y_1 \vee \ell_3 \vee y_2) \wedge (\neg y_2 \vee \ell_4 \vee \ell_5)$$

The y_i act as a relay: a chain of "if nothing so far was true, the truth must still be ahead".

WHY THE SPLITTING GADGET WORKS

$$(\ell_1 \vee \ell_2 \vee y_1) \wedge (\neg y_1 \vee \ell_3 \vee y_2) \wedge (\neg y_2 \vee \ell_4 \vee \ell_5)$$

($\Rightarrow$) If some ℓ_i is true, set the y's to hand the baton to its clause: e.g. if ℓ_3 is true, set $y_1 = 1, y_2 = 0$ — all three clauses satisfied.

($\Leftarrow$) If *all* ℓ_i are false, the first clause forces $y_1 = 1$, then the second forces $y_2 = 1$, and the third clause $(\neg y_2 \vee \ell_4 \vee \ell_5)$ dies. No assignment of the y's can help.

WHY THE SPLITTING GADGET WORKS

A clause of length k becomes $k-2$ clauses with $k-3$ new variables: the formula grows only linearly. Polynomial-time translation. ✓

> **COROLLARY** 3SAT is NP-complete.

THE PLAN FROM HERE

With one hard problem in hand, hardness spreads by reductions — each arrow is a polynomial-time translation:

$$\text{SAT} \leq_p \text{3SAT} \leq_p \text{Independent Set} \leq_p \begin{cases} \text{Vertex Cover} \\ \text{Clique} \end{cases}$$

Karp (1972) showed 21 problems NP-complete this way; today there are thousands.

3SAT $\leq_p$ INDEPENDENT SET

TARGET: INDEPENDENT SET

Problem **Independent Set:** given a graph G and integer k, does G have a set of $\geq k$ vertices with no edge between any two of them?

On trees: linear-time DP (Week 3). In general graphs: we now show it's NP-complete.

We must translate an arbitrary 3SAT formula φ with m clauses into a graph G_φ and a number k so that

$$\varphi \text{ satisfiable} \iff G_\varphi \text{ has an independent set of size } k.$$

Logic in, graph out. This is where gadgets earn their name.

THE CONSTRUCTION: A TRIANGLE PER CLAUSE

- For each clause, create a **triangle** with one vertex per literal occurrence.
- Triangle edges force a choice: an independent set can pick *at most one* vertex per triangle.
- Add a **conflict edge** between every pair of vertices labelled x and $\neg x$ — you cannot commit to both a literal and its negation.

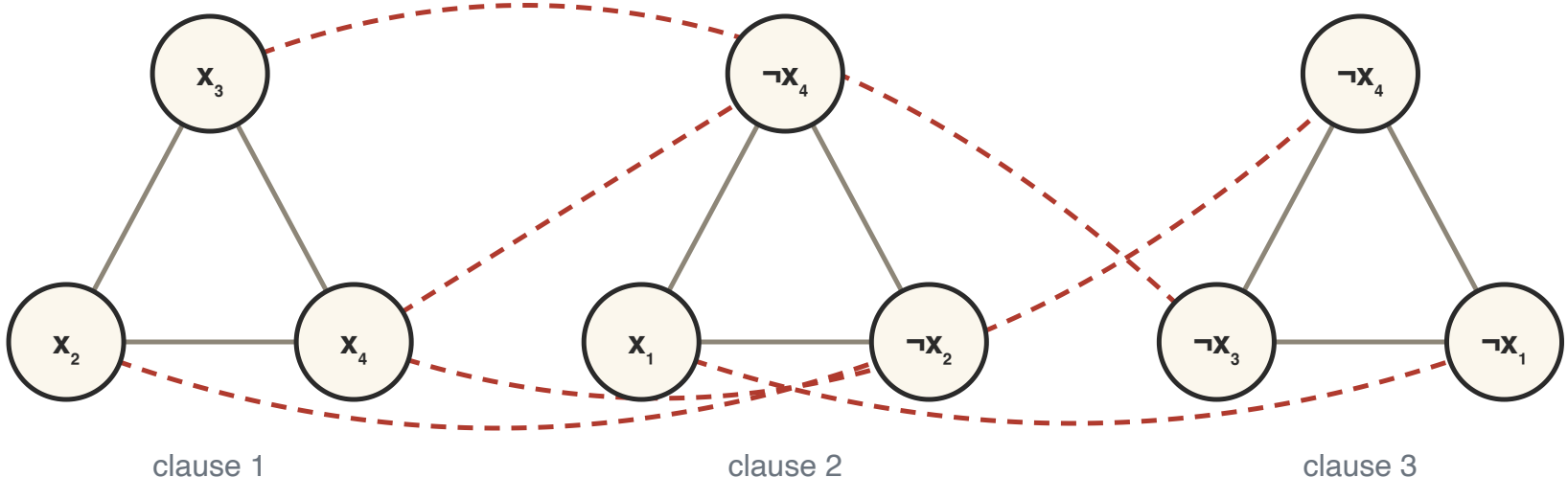
THE CONSTRUCTION: A TRIANGLE PER CLAUSE

- Set $k = m$, the number of clauses: to reach size m, the set must take *exactly one* vertex from *every* triangle.

An independent set of size m = a conflict-free choice of one literal per clause = a satisfying assignment. Let's see it on our Week 2 formula.

WORKED EXAMPLE: THE WEEK 2 FORMULA

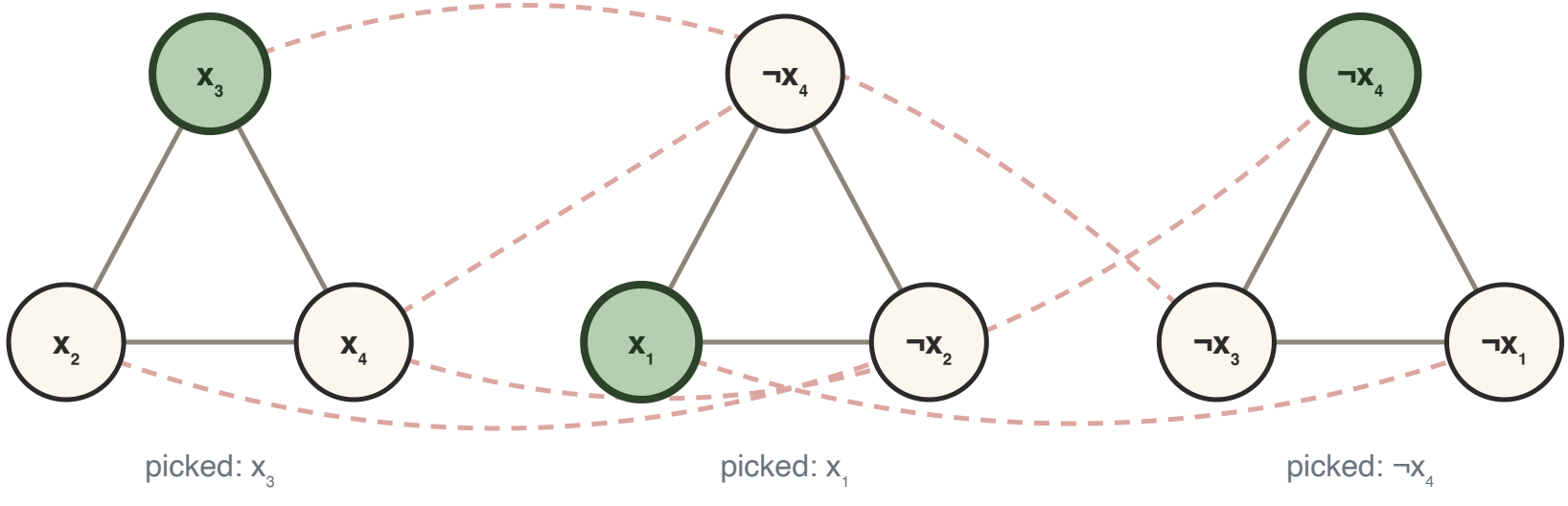
$$(x_3 \vee x_2 \vee x_4) \wedge (\neg x_4 \vee x_1 \vee \neg x_2) \wedge (\neg x_3 \vee \neg x_4 \vee \neg x_1) \quad k = 3$$



One triangle per clause (grey edges); dashed red conflict edges join x to $\neg x$.

FINDING THE INDEPENDENT SET

Week 2's satisfying assignment $x_1 = 1,\ x_2 = 0,\ x_3 = 1,\ x_4 = 0$ picks one true literal per triangle:



$\{x_3,\ x_1,\ \neg x_4\}$: one per triangle, no conflict edge inside — an independent set of size $3 = m$.

CORRECTNESS ($\Rightarrow$): SATISFIABLE $\Rightarrow$ IS OF SIZE m

- Fix a satisfying assignment. Every clause has at least one true literal; pick *one* true literal's vertex in each triangle. That's m vertices.
- No triangle edge inside the set: we took one vertex per triangle.
- No conflict edge inside the set: a conflict edge joins x and $\neg x$, and a single assignment cannot make both true.
- So the chosen vertices form an independent set of size m. ✓

CORRECTNESS ($\Leftarrow$): IS OF SIZE $m \Rightarrow$ SATISFIABLE

- Let S be independent with $|S| = m$. Each triangle contributes at most one vertex, and there are m triangles — so *exactly one per triangle*.
- Read off an assignment: if a vertex labelled x_i is in S, set $x_i = 1$; if labelled $\neg x_i$, set $x_i = 0$.
- Consistent: S contains no conflict edge, so it never selects both x_i and $\neg x_i$. Variables mentioned by no selected vertex: set arbitrarily.

CORRECTNESS ($\Leftarrow$): IS OF SIZE $m \Rightarrow$ SATISFIABLE

- Every clause's triangle has a selected vertex whose literal is true — every clause is satisfied. ✓

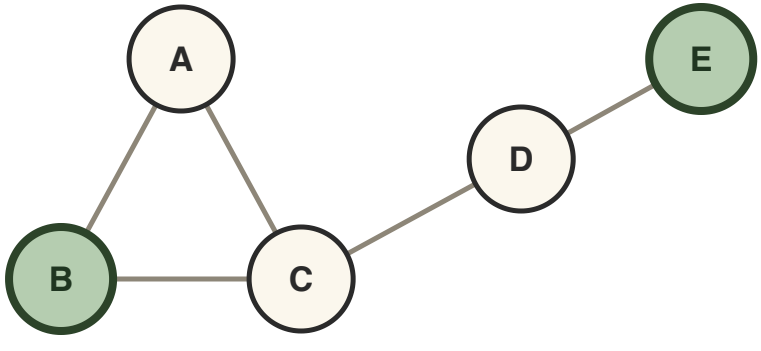
THEOREM Independent Set is NP-complete. (In NP: the set is the certificate. NP-hard: 3SAT $\leq_p$ IS above; the graph has $3m$ vertices — polynomial size.)

INDEPENDENT SET $\leftrightarrow$ VERTEX COVER $\leftrightarrow$ CLIQUE

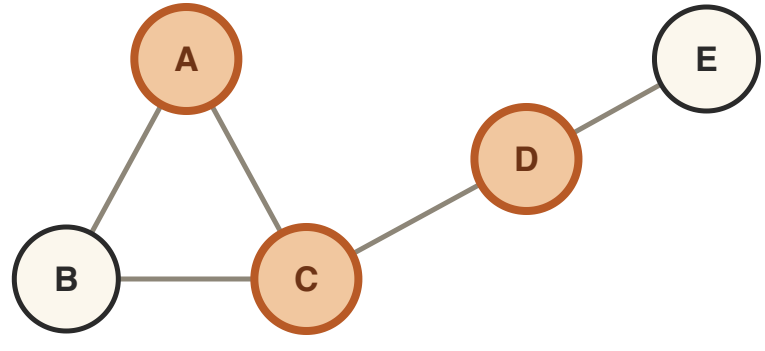
IS AND VC: TWO VIEWS

FACT S is an independent set of $G = (V, E) \iff$ $V \setminus S$ is a vertex cover of G.

Proof: S independent $\iff$ no edge has both endpoints in S $\iff$ every edge has an endpoint in $V \setminus S$ $\iff$ $V \setminus S$ covers all edges. $\square$



independent set S = {B, E}



vertex cover V∖S = {A, C, D}

The same graph: the complement of an independent set covers every edge.

SO VERTEX COVER IS NP-COMPLETE TOO

The fact gives an instant two-way translation:

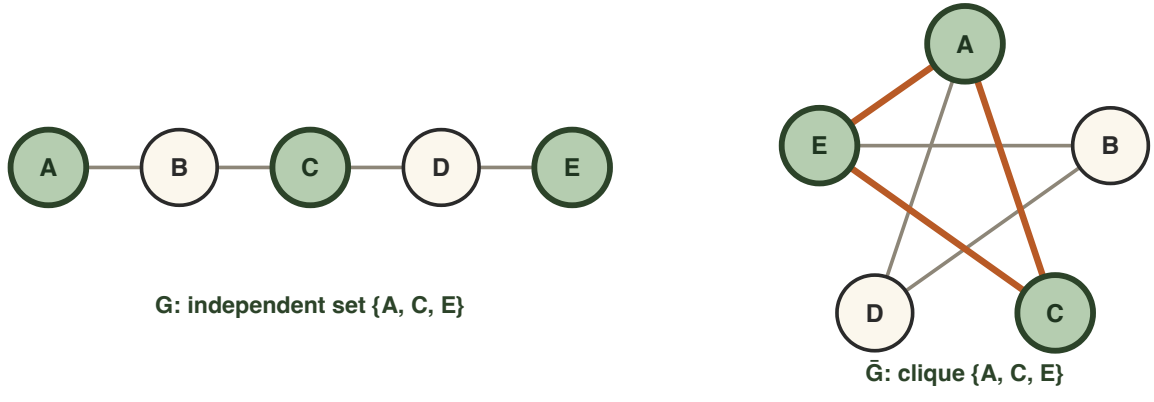
$$G \text{ has an independent set of size } \geq k \iff G \text{ has a vertex cover of size } \leq n - k.$$

- The "reduction" f barely does anything: same graph, replace k by $n - k$. Reductions can be this cheap.
- IS $\leq_p$ VC, hence VC is NP-hard; the cover itself is a certificate, so VC $\in$ NP.

COROLLARY Vertex Cover is NP-complete.

CLIQUE: FLIP THE EDGES

FACT S is an independent set of $G \iff S$ is a clique of the complement graph $\bar{G}$ (same vertices; edges exactly where G has none).



Non-edges of G become edges of $\bar{G}$: mutual strangers become mutual friends.

Reduction: map (G, k) to $(\bar{G}, k)$ — computable in polynomial time. So Clique is NP-complete as well.

THREE PROBLEMS, ONE HARDNESS

Problem	Question	Status
Independent Set	$\geq k$ mutually non-adjacent vertices?	NP-complete
Vertex Cover	$\leq k$ vertices touching all edges?	NP-complete
Clique	$\geq k$ mutually adjacent vertices?	NP-complete

THREE PROBLEMS, ONE HARDNESS

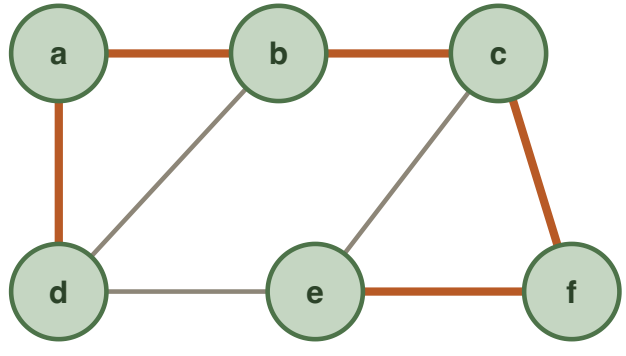
They are the *same problem* wearing three costumes — complementing the set, or complementing the edges, translates between them for free.

This is the power of $\leq_p$: hardness proved once radiates outward.

HAMILTONIAN PATH

DEFINITION AND EXAMPLE

Definition A **Hamiltonian path** in G visits every *vertex* exactly once. Decision problem: does G have one?



A Hamiltonian path $d \to a \to b \to c \to f \to e$ (orange).

The certificate is the ordering — easy to verify, so the problem is in NP.

HAMILTONIAN PATH IS NP-COMPLETE

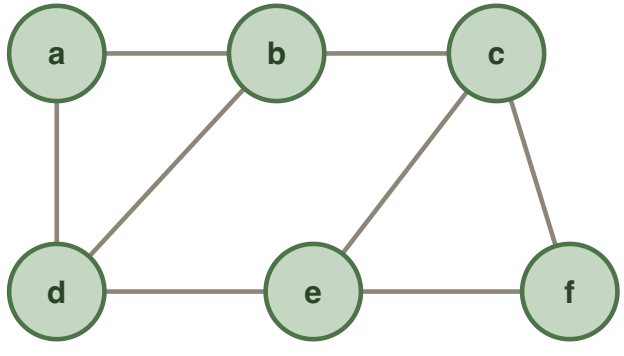
THEOREM Hamiltonian Path is NP-complete (via 3SAT $\leq_p$ Hamiltonian Path).

We skip the construction. Each variable becomes a chain of vertices, traversed left-to-right for *true* and right-to-left for *false*; each clause becomes a vertex that can be visited only by a chain going the satisfying way.

The lesson isn't the wiring: NP-completeness proofs are *constructions*, and gadget engineering is a real skill.

EULERIAN PATH

DEFINITION An **Eulerian path** in G uses every *edge* exactly once (vertices may repeat). Decision problem: does G have one?



The graph from the Hamiltonian path example.

It has a Hamiltonian path. Does it have an Eulerian path?

CONTRAST: EULERIAN PATH IS EASY

	Eulerian path	Hamiltonian path
visits every…	*edge* exactly once	*vertex* exactly once
complexity	polynomial (linear!)	NP-complete

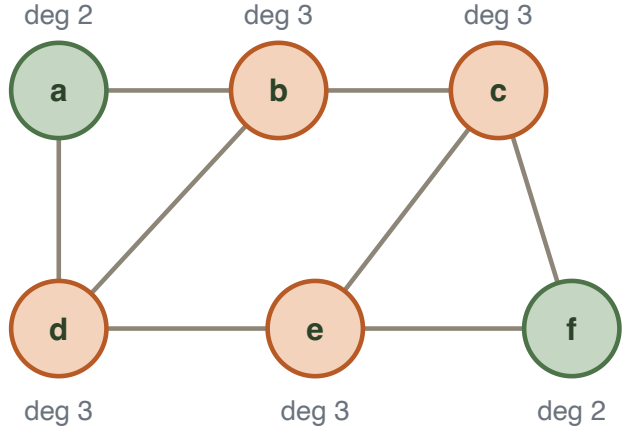
CONTRAST: EULERIAN PATH IS EASY

Why? Eulerian path has a clean characterisation — connected + at most two odd-degree vertices. For Hamiltonian path, no such characterisation is known (or expected).

Euler's condition is just degree counting; Hamiltonicity refuses to localise.

Moral: problem statements one word apart can sit on opposite sides of the P / NP-complete divide. Only proofs — algorithms or reductions — tell you which.

BACK TO OUR GRAPH



b, c, d, e have odd degree (orange).

- Four odd-degree vertices, so **no** Eulerian path — found by counting degrees.
- Yet $d \to a \to b \to c \to f \to e$ is a Hamiltonian path. The two problems really are different.

SUBSET SUM

SUBSET SUM

Problem **Subset Sum:** given positive integers $a_1, \ldots, a_n$ and a target T, is there a subset of the a_i that sums to *exactly* T?

Example: $\{3, 5, 8, 13\}$ with $T = 21$. Yes: $8 + 13 = 21$.

In NP: the subset is the certificate. Add it up and compare with T.

Our first problem today with *numbers* in it — and the numbers will matter.

A DP FOR SUBSET SUM

Let $R[i][s]$ be true if some subset of $a_1, \ldots, a_i$ sums to exactly s.

- Base: $R[0][s]$ is true only for $s = 0$ (the empty subset).
- Either a_i is left out, or it is used ($s \geq a_i$):

$$R[i][s] = R[i-1][s] \ \vee \ R[i-1][s - a_i]$$

- Answer: $R[n][T]$. There are $(n+1)(T+1)$ entries, each $O(1)$: time $O(nT)$.

A DP FOR SUBSET SUM

On $\{3, 5, 8, 13\}$ with $T = 21$: the sums $s \leq 21$ with $R[i][s]$ true.

i	a_i	Reachable sums
0		0
1	3	$0, 3$
2	5	$0, 3, 5, 8$
3	8	$0, 3, 5, 8, 11, 13, 16$
4	13	$0, 3, 5, 8, 11, 13, 16, 18, 21$

Each row is the previous row, together with the previous row shifted up by a_i.

A PARADOX?

PUZZLE We are about to prove that Subset Sum is NP-complete. But we just solved it in $O(nT)$ time — a polynomial! Have we proved P $=$ NP?

Hint: what is the *size of the input*?

INPUT SIZE COUNTS BITS

- Input size is measured in bits, and the number T takes only about $\log_2 T$ of them.
- So $O(nT) = O(n \cdot 2^{\log_2 T})$ is **exponential** in the length of T.
- Concretely: 100 numbers of 100 bits each is a $10{,}000$-bit input, but the table has about $100 \cdot 2^{100} \approx 10^{32}$ entries.

INPUT SIZE COUNTS BITS

Definition An algorithm is **pseudo-polynomial** if its running time is polynomial in n and the *values* of the numbers — equivalently, polynomial if the numbers were written in unary.

The DP is fast exactly when the numbers are small: if $T \leq n^c$, then $O(nT)$ is polynomial.

So a reduction 3SAT $\leq_p$ Subset Sum *must* produce numbers with polynomially many digits but exponentially large values. That is our design constraint.

DESIGNING THE REDUCTION

Given a 3SAT formula φ with variables $x_1, \ldots, x_n$ and clauses $C_1, \ldots, C_m$, build numbers and a target T so that

$$\varphi \text{ satisfiable} \iff \text{some subset sums to exactly } T.$$

- Choosing a subset must encode a truth assignment: for each of the n variables, *exactly one* of $x_i, \neg x_i$.
- Hitting T must certify that each of the m clauses has a true literal.

That is $n + m$ constraints. But Subset Sum gives us only *one* equation…

DIGITS AS COUNTERS

Trick: one equation between big numbers can hide many equations between digits.

- Give each constraint its own decimal column.
- Put a 1 in a number's column if choosing that number should count toward that constraint.

	COL A	COL B	COL C
110	1	1	0
011	0	1	1
sum 121	1	2	1

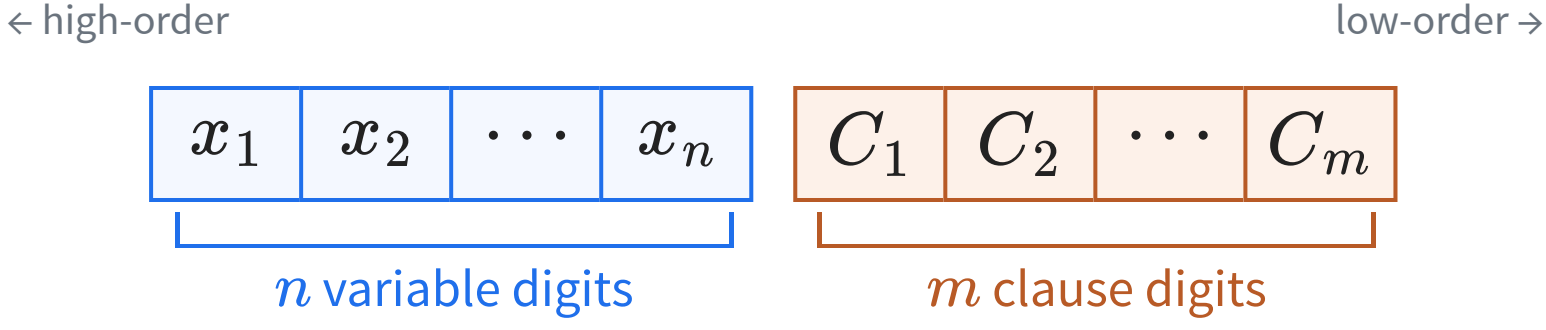
DIGITS AS COUNTERS

Each digit of the sum *counts* the chosen numbers marked in that column — as long as no column reaches 10 and carries into its neighbour.

KEY IDEA With no carries, "the subset sums to T" means "in *every* column, the count equals T's digit." That is $n + m$ checks packed into one equation.

LAYOUT OF EVERY NUMBER

Every number we build, and T, has exactly $n + m$ decimal digits:



- Digit x_i: does this number set the value of x_i?
- Digit C_j: does this number help satisfy C_j?
- Only $n + m$ digits long, yet values up to 10^{n+m} — exactly the design constraint.

ONE COLUMN PER VARIABLE

- For each variable x_i, make two numbers: t_i ("x_i is true") and f_i ("x_i is false").
- Both have a 1 in column x_i and a 0 in every other variable column.
- T has a 1 in column x_i.

Column x_i sums to 1 $\iff$ *exactly one* of t_i, f_i is chosen $\iff$ x_i gets exactly one truth value.

SO FAR t_i and f_i look identical. They will differ in the *clause* columns, which record what each choice makes true.

ONE COLUMN PER CLAUSE

- Put a 1 in column C_j of t_i if x_i appears in C_j, and of f_i if $\neg x_i$ appears in C_j.
- Then column C_j sums to the number of *true literals* in C_j: $0, 1, 2$ or 3.

OBSTRUCTION A clause needs *at least one* true literal, but Subset Sum only tests *equality*. Which digit should T have in column C_j?

SLACK CLOSES THE GAP

- For each clause C_j, add two **slack numbers** s_j, s'_j, each with a single 1 in column C_j.
- Give T the digit 3 in column C_j.
- $1, 2$ or 3 true literals: top up with $2, 1$ or 0 slack numbers.
- 0 true literals: slack adds at most $2 < 3$, so the column cannot reach 3 — exactly the unsatisfied case.

Largest column sum: $3 + 2 = 5 \leq 9$, so nothing ever carries.

WORKED EXAMPLE

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

	x_1	x_2	x_3	C_1	C_2
t_1	1	0	0	1	0
f_1	1	0	0	0	1
t_2	0	1	0	0	1
f_2	0	1	0	1	0
t_3	0	0	1	1	1
f_3	0	0	1	0	0
s_1, s_1'	0	0	0	1	0
s_2, s_2'	0	0	0	0	1
T	1	1	1	3	3

Each row is a base-10 number:
$t_1 = 10010$, $f_3 = 100$,
$T = 11133$.

In general: $2n + 2m$ numbers of $n + m$ digits each — polynomial size.

WORKED EXAMPLE

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

$x_1 = 1,\ x_2 = 0,\ x_3 = 1$ picks t_1, f_2, t_3; then slack tops up C_2

	x_1	x_2	x_3	C_1	C_2
t_1	1	0	0	1	0
f_1	1	0	0	0	1
t_2	0	1	0	0	1
f_2	0	1	0	1	0
t_3	0	0	1	1	1
f_3	0	0	1	0	0
s_1, s_1'	0	0	0	1	0
s_2, s_2'	0	0	0	0	1
T	1	1	1	3	3

C_1: three true literals, no slack.

C_2: one true literal, plus s_2 and s_2'.

$$10010 + 1010 + 111 + 1 + 1 = 11133 = T \checkmark$$

WORKED EXAMPLE

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$$

$x_1 = 1,\ x_2 = 0,\ x_3 = 0$ picks t_1, f_2, f_3, but C_2 is unsatisfied

	x_1	x_2	x_3	C_1	C_2
t_1	1	0	0	1	0
f_1	1	0	0	0	1
t_2	0	1	0	0	1
f_2	0	1	0	1	0
t_3	0	0	1	1	1
f_3	0	0	1	0	0
s_1, s_1'	0	0	0	1	0
s_2, s_2'	0	0	0	0	1
T	1	1	1	3	3

C_2: no true literal. Column C_2 gets 0 from t_1, f_2, f_3.

Even with both s_2, s_2' it reaches only $2 \neq 3$. No subset hits T with this choice.

CORRECTNESS ($\Rightarrow$)

Given a satisfying assignment:

- Choose t_i if $x_i = 1$, else f_i. Every variable column sums to 1.
- Clause C_j has $\ell_j \in \{1, 2, 3\}$ true literals, so column C_j gets ℓ_j; add $3 - \ell_j \leq 2$ of its slack numbers.
- Every column matches T's digit and nothing carries, so the chosen numbers sum to T. ✓

CORRECTNESS ($\Leftarrow$)

Given a subset that sums to T:

- Every column sums to at most 5, so nothing carries: each column of the subset matches T's digit.
- Column x_i is 1: exactly one of t_i, f_i is chosen. Set x_i accordingly.
- Column C_j is 3 and slack supplies at most 2, so some chosen t_i or f_i has a 1 there: a true literal of C_j. ✓

THE PARADOX, RESOLVED

THEOREM Subset Sum is NP-complete. (In NP: the subset is the certificate. NP-hard: 3SAT $\leq_p$ Subset Sum above.)

The reduction's target $T = 1 \cdots 1\, 3 \cdots 3$ has $n + m$ digits: short to write down, but its *value* is about 10^{n+m}.

On these instances the $O(nT)$ DP takes exponential time. No contradiction — just as the design constraint predicted.

WEAK AND STRONG HARDNESS

- **Weakly NP-hard:** hard only when the numbers are huge; a pseudo-polynomial algorithm exists. Subset Sum, Knapsack.
- **Strongly NP-hard:** hard even when every number is polynomially bounded — or there are no numbers at all. 3SAT, Independent Set, Vertex Cover, Hamiltonian path.

Practical moral: when an NP-hard problem's numbers are small, look for a DP over the values.

MEET IN THE MIDDLE

Back to *solving* Subset Sum: given n numbers and a target T, brute force tries all 2^n subsets. Can we do better when the numbers are large?

- Split the numbers into two halves A and B, each of size $n/2$.
- Any subset is a part of A plus a part of B, so its sum is $x + y$, with x a subset sum of A and y a subset sum of B.
- Let L_A, L_B list all subset sums of each half: $2^{n/2}$ entries each.

New question: is there $x \in L_A$ with $T - x \in L_B$?

MEET IN THE MIDDLE

1. List L_A and L_B.
2. Sort L_B.
3. For each $x \in L_A$, binary-search L_B for $T - x$.

Sorting $2^{n/2}$ numbers and doing $2^{n/2}$ binary searches costs $O(2^{n/2} \cdot n)$ time: the *square root* of brute force.

The price is $\Omega(2^{n/2})$ space for the lists.

Horowitz and Sahni, “Computing Partitions with Applications to the Knapsack Problem”, JACM 1974.

MEET IN THE MIDDLE

On $\{3, 5, 8, 13\}$ with $T = 21$:

$$A = \{3, 5\}: \quad L_A = \{0, 3, 5, 8\}$$
$$B = \{8, 13\}: \quad L_B = \{0, 8, 13, 21\}$$

$x \in L_A$	$T - x$	IN L_B?
0	21	yes: $8 + 13$
3	18	no
5	16	no
8	13	yes: $3 + 5 + 13$

TWO WAYS TO BEAT BRUTE FORCE

ALGORITHM	TIME	WINS WHEN...
DP over sums	$O(nT)$	the numbers are small
Meet in the middle	$O(2^{n/2} \cdot n)$	there are few numbers

Can we get $2^{o(n)}$? Not under ETH. Our reduction turns a 3SAT formula into $2n + 2m$ numbers, and we may assume $m = O(n)$ clauses (the sparsification lemma). So a $2^{o(n)}$ Subset Sum algorithm would solve 3SAT in $2^{o(n)}$ time.

BELOW $2^{n/2}$?

So the question is the constant: is $\frac{1}{2}$ the right exponent?

- For 50 years, worst-case progress saved only space, down to $2^{n/4}$ (Schroeppel–Shamir 1981), or polynomial factors in time.
- On *random* instances, $2^{0.291n}$ was known (Becker–Coron–Joux 2011) — but not for every input.

BELOW $2^{n/2}$?

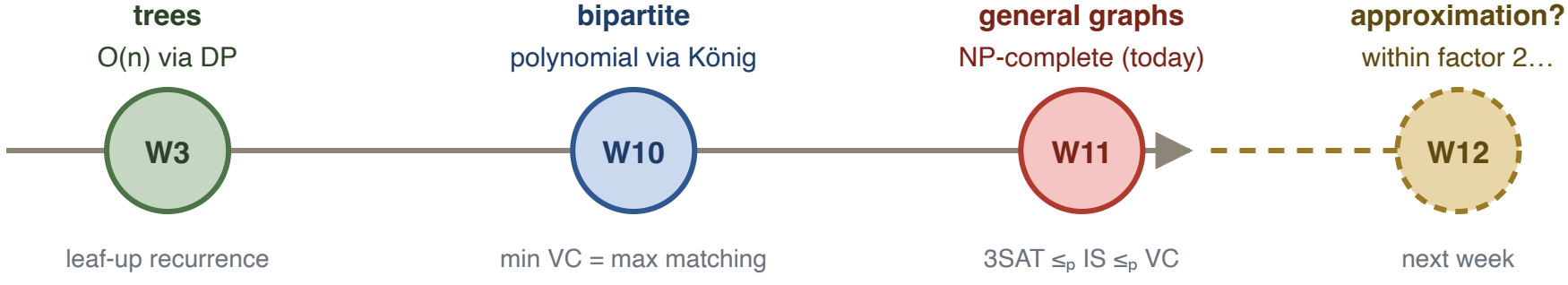
OCTOBER 2026 A randomized algorithm for Subset Sum running in time $O(2^{0.49n})$ on *every* input, found by an internal OpenAI model [OAI26].

It is a preprint: not yet peer-reviewed or formally verified. Read it as a claim, and watch this space.

[OAI26] OpenAI, "Subset Sum in Time $O(2^{0.49n})$", preprint, October 4, 2026.

THE VERTEX COVER STORY

ONE PROBLEM, A WHOLE COURSE



Minimum vertex cover across the course: the input class decides everything.

- **Trees (W3):** a linear-time DP — hard problems can have easy special cases.
- **Bipartite (W10):** LP duality and König make it a matching problem.
- **General (today):** NP-complete — no polynomial algorithm unless P = NP.

MATCHING VS VERTEX COVER

Graphs	Max matching	Min vertex cover
Trees	tree DP	tree DP
Bipartite	max flow	König: = max matching
General	**polynomial** (Edmonds)	**NP-complete**

On bipartite graphs König made them one problem. On general graphs they split across the P / NP-complete divide.

Edmonds' blossom algorithm: “Paths, Trees, and Flowers”, 1965 — the same paper argued that “polynomial time” is the right meaning of *efficient*.

MATCHING VS VERTEX COVER

- Weak duality survives on every graph: each matching edge needs its own cover vertex, so $|M| \leq |C|$ for any matching M and cover C.
- Equality does not: on a triangle the maximum matching has 1 edge, but every vertex cover needs 2 vertices.
- How large can the gap get? Next week: never more than a factor of 2 — and that bound becomes an algorithm.

SO… WHAT NOW?

NP-completeness is not a licence to give up. It redirects the question:

- Exploit structure: is your instance a tree? bipartite? small? (W3, W10)
- Beat brute force without reaching polynomial: $O(2^{0.39n})$ for 3SAT.
- If we can't solve it exactly, how close can we get?

Next week: a polynomial-time algorithm that always finds a vertex cover at most **twice** the minimum — using LP relaxations (W9) and duality (W10). The vertex cover arc gets its final chapter.

TAKEAWAYS

- **NP** = short certificates + fast verifiers. Finding vs checking is the whole question.
- $A \leq_p B$: easiness flows backward, hardness flows forward. Mind the arrow's direction.
- **Cook–Levin** seeds the theory: SAT is NP-complete because computation is local and locality is clauses.
- **Gadgets** (clause splitting, triangles, complements) carry hardness from logic into graphs and numbers.
- SAT, 3SAT, IS, VC, Clique, Hamiltonian path, Subset Sum: all NP-complete — one polynomial algorithm for any of them would crack them all.